

ClimateTwin India: Architectural Blueprint

A step-by-step roadmap for manual monorepo development from scratch

1. Project Planning & Conception

ClimateTwin India is a digital twin command center designed to monitor regional climate data, model localized risk indices (floods, droughts, heatwaves, and UHI index), and run parametric simulations for disaster management and Clean Energy forecasting. The codebase is organized as a type-safe TypeScript/Python monorepo.

2. Tech Stack Selection

Layer	Technologies Used
Monorepo Orchestrator	PNPM Workspaces (manages dependencies, builds, and library linkings)
Database Layer	PostgreSQL (Relational data storage) + Drizzle ORM (schema/migrations)
Backend REST APIs	Python FastAPI (async routes, Pydantic type safety, fast runtime)
Frontend Web App	React v18 + Vite (fast builder) + TailwindCSS (styling layout)
Client Typings	Orval (Codegen typescript client hooks & zod validators from OpenAPI spec)
AI / LLM Integration	Google Gemini 2.5 Flash API (live telemetry context-aware advice)

3. System Architecture & E2E Data Flow

Data flow pathway:

[React UI (Client)] ^ (TanStack React Query GET / POST) v [FastAPI Backend (Python Router)] ^ (SQLAlchemy AsyncSession) v [PostgreSQL Database (Current Telemetry)] +---> (Gemini API Prompt Injection: Telemetry + User Input) v [Generative AI Reply (Dynamic Context-Aware Recommendation)]

4. Workspace Folder Structure Layout

```
climatetwin-root/
|-- package.json          # Workspaces settings & root typecheck runner
|-- pnpm-workspace.yaml   # Monorepo packages mapping
|-- lib/
|  |-- api-spec/          # OpenAPI spec repository (openapi.yaml)
|  |-- api-client-react/  # Codegen react-query fetch hooks
|  |-- api-zod/           # Codegen zod validation schemas
|-- backend/
|  |-- app/
|  |  |-- api/endpoints/  # FastAPI endpoint controllers (climate, scenarios, energy)
|  |  |-- database/       # PostgreSQL db connection and session helpers
|  |  |-- models/         # SQLAlchemy database model models (base.py)
|  |  |-- schemas/        # Pydantic schemas validation models
|  |-- verify_endpoints.py # Backend API validation suite
|  |-- requirements.txt    # Python dependency file
|-- artifacts/
|  |-- climate-twin/      # Vite + React frontend dashboard web app
|  |  |-- src/
|  |  |  |-- components/  # Shared components (layout, sidebar, theme-provider)
|  |  |  |-- pages/       # Dashboard modules (dashboard, simulator, energy, disasters)
|  |  |-- vite.config.ts  # Vite builder config with proxy settings
```

5. Database Schema & Models Configuration

The PostgreSQL instance stores current climate snapshots and command summary stats. SQLAlchemy schemas map these models to async sessions.

Table: climate_snapshots	Fields & Types
id	Integer, Primary Key (serial)
temperature, humidity	Float, representing active sensors values
aqi, forest_fire_risk	Integer indices
recorded_at	DateTime, defaults to timezone.utc

6. Step-by-Step Development Roadmap

* Step 1: Monorepo Bootstrapping

Initialize PNPM workspaces. Define the root package.json and pnpm-workspace.yaml. Set up TypeScript configuration files and base path mappings.

* Step 2: Database Setup & Seed

Provision PostgreSQL db. Create database schema tables via Drizzle/SQLAlchemy. Write and execute seeds script to populate initial regional baseline climate data.

* Step 3: OpenAPI Spec Drafting

Draft openapi.yaml defining all required endpoints (/climate/current, /scenarios/simulate, /agriculture/chat, /energy/dashboard, /reports/download). Run client codegen compiling React hooks.

* Step 4: FastAPI Backend Controllers

Implement models, schemas, and endpoint routers in Python. Setup async db connection sessions. Write parametric simulation formulas, AI chatbot controllers, and FPDF PDF reports streams.

* Step 5: React Frontend Assembly

Initialize Vite React app inside workspaces. Create Sidebar layout with router wrapper. Build individual pages (dashboard, disasters, agriculture, energy, simulator) hooking up React Query.

* Step 6: Dark/Light Theme Switching

Construct stateful ThemeProvider persisting theme (dark/light) in localStorage. Add class overrides in index.css. Create header toggle switcher button.

* Step 7: Global AI Chatbot Integration

Create global-chatbot.tsx widget. Setup backend router calling Google Gemini API injecting db telemetry context. Mount widget inside layout container.

7. API Integrations Specifications

Google Gemini API	POST https://generativelanguage.googleapis.com/... Generates telemetry-aware advisory recommendations. Auth
Telemetry Sensor API	GET /api/climate/current. Fetches local PostgreSQL data, applies random jitter to simulate active sensors.
Reports Downloader	GET /api/reports/download. Compiles active simulation inputs/outputs into a binary PDF file via FPDF.

8. Verification, Testing & Deployment

Verification: Run full monorepo typecheck to confirm compilation. Execute verify_endpoints.py to validate API routers.
Deployment: Dockerize backend API app. Deploy database on RDS. Build and host React static files on S3/Vercel with Vite proxy configured.

9. End-to-End Development Checklist

Task Checklist Matrix:

[] Stage 1: Setup pnpm-workspace.yaml & root configurations.
[] Stage 2: Provision PostgreSQL database and run SQL seed scripts.
[] Stage 3: Write openapi.yaml contract and execute client compiler codegen.
[] Stage 4: Build FastAPI async routes & setup database models.
[] Stage 5: Design Vite+React layout & dashboard modules.
[] Stage 6: Integrate FPDF engine to generate binary reports.
[] Stage 7: Implement ThemeProvider context & light/dark theme variables.
[] Stage 8: Code Gemini API client integration and mount floating chatbot.
[] Stage 9: Execute verify_endpoints.py validation suite & compile bundle.